Καταρχάς, παρουσιάζεται η διεπαφή:

```
public interface Deck {
    public void shuffle();
    public boolean empty();
    public Card pickTop();
}
```

Κώδικας 14.12 *Η διεπαφή Deck*

Οι μέθοδοι shuffle και pickTop απαιτούνται άμεσα από την εκφώνηση. Η μέθοδος empty που ελέγχει αν υπάρχει ή όχι στην τράπουλα κάρτα είναι γενικότερα χρήσιμη. Στη συνέχεια δίνεται η κλάση Deck52 που αναπαριστά τράπουλες των 52 καρτών.

```
import java.util.Random;

public class Deck52 implements Deck {

    final Card[] deck;
    private int cardsInDeck;
    private static final int DECKSIZE = 52;

    public Deck52() {
        deck = new Card[DECKSIZE];
        cardsInDeck = 0;
        for (CardRank r : CardRank.values()) {
            for (Suit s : Suit.values()) {
                deck[cardsInDeck++] = new Card(r, s);
            }
        }
    }

    @Override
    public void shuffle() {
        Random r = new Random();

        for (int i = 0; i < deck.length; i++) {
            int rIdx = r.nextInt(deck.length);
            Card tmp = deck[i];
            deck[i] = deck[rIdx];
            deck[rIdx] = tmp;
        }
    }

    @Override
    public boolean empty() {
        return cardsInDeck == 0;
    }

    @Override
    public Card pickTop() {
        cardsInDeck--;
        return deck[DECKSIZE - cardsInDeck - 1];
    }

    @Override
    public String toString() {
        if (cardsInDeck == 0) {
            return "Empty Deck";
        }
        StringBuilder rVal = new StringBuilder();
```

```
            for (int i = DECKSIZE - cardsInDeck; i < DECKSIZE; i++) {
                if (i > DECKSIZE - cardsInDeck && i % 4 == 0) {
                    rVal.append("\n");
                }
                rVal.append(deck[i] + " ");
            }
            return rVal.toString();
        }
    }
```

Κώδικας 14.13 *Η κλάση Deck52*

Παρόμοια είναι και η κλάση Deck32 που αναπαριστά τράπουλες των 32 καρτών.

```
    import java.util.Random;

    public class Deck32 implements Deck {

        private final Card[] deck;
        private int cardsInDeck;
        private static final int DECKSIZE = 32;

        public Deck32() {
            deck = new Card[DECKSIZE];
            cardsInDeck = 0;
            for (int i = 5; i < 13; i++) {
                CardRank r = CardRank.values()[i];
                for (Suit s : Suit.values()) {
                    deck[cardsInDeck++] = new Card(r, s);
                }
            }
        }

        @Override
        public void shuffle() {
            Random r = new Random();

            for (int i = 0; i < deck.length; i++) {
                int rIdx = r.nextInt(deck.length);
                Card tmp = deck[i];
                deck[i] = deck[rIdx];
                deck[rIdx] = tmp;
            }
        }

        @Override
        public boolean empty() {
            return cardsInDeck == 0;
        }

        @Override
        public Card pickTop() {
            cardsInDeck--;
            return deck[DECKSIZE - cardsInDeck - 1];
        }

        @Override
        public String toString() {
            if (cardsInDeck == 0) {
                return "Empty Deck";
            }
            StringBuilder rVal = new StringBuilder();
```

```
            for (int i = DECKSIZE - cardsInDeck; i < DECKSIZE; i++) {
                if (i > DECKSIZE - cardsInDeck && i % 4 == 0) {
                    rVal.append("\n");
                }
                rVal.append(deck[i] + " ");
            }
            return rVal.toString();
        }
    }
```

Κώδικας 14.14 *Η κλάση Deck32*

Στον κώδικα 14.15 δίνεται και η εφαρμογή που ζητείται από την εκφώνηση.

```
    public class App {
        public static void main(String[] args) {
            Deck[] decks=new Deck[2];
            decks[0]=new Deck32();
            decks[1]=new Deck52();
            for (Deck deck:decks) {
                deck.shuffle();
                System.out.println(deck.pickTop());
                System.out.println("------------------------------");
                System.out.println(deck);
                System.out.println("------------------------------");
            }
        }
    }
```

Κώδικας 14.15 *Η ζητούμενη εφαρμογή*

14.5.2 Τζόγος στου John

Στη συγκέντρωση που διοργάνωσε ο John Beker μαζεύτηκαν 14 πρόσωπα. Σε κάποιους από τους καλεσμένους αρέσει να παίζουν ζάρια. Ο John έχει 3 ζάρια. Κάποια ζάρια είναι κανονικά και φέρνουν κάθε αποτέλεσμα από 1 έως 6 με πιθανότητα 1/6, κάποια άλλα όμως δεν φέρνουν ποτέ 5 και φέρνουν 6 με πιθανότητα 1/3. Τέλος, υπάρχουν και ζάρια που δεν φέρνουν 4 ποτέ αλλά φέρνουν 5 με πιθανότητα 1/3.

Επίσης, κάποιοι από τους καλεσμένους παίζουν χαρτιά.

Στους καλεσμένους του John συμπεριλαμβάνονται κάποιοι που παίζουν αποκλειστικά ζάρια, κάποιοι που παίζουν αποκλειστικά χαρτιά αλλά και κάποιοι που παίζουν και χαρτιά και ζάρια.

Σε μια παρτίδα ζαριών συμμετέχουν 2 παίκτες. Ρίχνουν εναλλάξ από 10 ζαριές. Κάθε φορά που οι 2 παίκτες ρίχνουν το ζάρι είτε ο ένας φέρνει μεγαλύτερο αποτέλεσμα από τον άλλον είτε φέρνουν το ίδιο αποτέλεσμα. Στην πρώτη περίπτωση ενημερώνεται αθροιστικά το σκορ του παίκτη που έφερε το μεγαλύτερο αποτέλεσμα. Στη δεύτερη περίπτωση δεν ενημερώνεται κανένα σκορ.

Σε μια παρτίδα χαρτιών συμμετέχουν όλοι οι διαθέσιμοι (χαρτο)παίκτες. Στην αρχή της παρτίδας ανακατεύουν την τράπουλα και στη συνέχεια τραβάνε από μια κάρτα εναλλάξ έως ότου τελειώσουν οι κάρτες. Στο τέλος της παρτίδας, ο κάθε παίκτης παρουσιάζει τα χαρτιά που έχει μαζέψει.

Ζητείται να αναπτυχθεί εφαρμογή που:

1. Βάζει κάθε πρόσωπο στη συγκέντρωση του John να συναντηθεί με κάθε άλλο πρόσωπο. Αν κατά τη συνάντηση είναι και οι 2 παίκτες ζαριού, τότε παίζουν μια παρτίδα ζάρια. Αν όμως κατά τη συνάντηση κάποιος είναι χαρτοπαίκτης, τότε εγγράφεται στην ομάδα των χαρτοπαικτών.
2. Όλοι οι εγγεγραμμένοι χαρτοπαίκτες συμμετέχουν σε μια παρτίδα χαρτιών.
3. Εμφανίζει το ονοματεπώνυμο και σκορ για κάθε παίκτη ζαριού κατά φθίνουσα σειρά των σκορ.
4. Νικητής στα χαρτιά ανακηρύσσεται αυτός που έχει μαζέψει τους περισσότερους άσους. Αν 2 ή περισσότεροι παίκτες έχουν τον ίδιο μέγιστο αριθμό άσων, τότε ανακηρύσσονται νικητές.

Λύση

Καταρχάς, θα πρέπει τώρα να σχεδιάσουμε μια κλάση για τα πρόσωπα που είναι παίκτες ζαριού, μια για όσους είναι χαρτοπαίκτες και μία για όσους παίζουν και χαρτιά και ζάρια. Και οι τρεις κλάσεις είναι παράγωγες της κλάσης Person. Επιπλέον, η κλάση των χαρτοπαικτών υλοποιεί τη διεπαφή CardPlayerI, η κλάση των παικτών ζαριού υλοποιεί τη διεπαφή DicePlayerI και η κλάση των προσώπων που είναι και χαρτοπαίκτες και παίκτες ζαριού υλοποιεί και την DicePlayerI και την CardPlayerI. Την τελευταία κλάση, την έχουμε υλοποιήσει ήδη (κώδικας 14.7).

```
import java.time.LocalDate;
import java.util.ArrayList;

public class CardPlayer extends Person implements CardPlayerI {

    ArrayList<Card> cardsInHand = new ArrayList<>();

    public CardPlayer(String id, String fName, String sName, LocalDate
birthday) {
        super(id, fName, sName, birthday);
    }

    @Override
    public Card pickTop(Deck deck) {
        Card card = deck.pickCard();
        cardsInHand.add(card);
        return card;
    }

    @Override
    public ArrayList<Card> openCards() {
        ArrayList<Card> rVal = new ArrayList<>(cardsInHand);
        cardsInHand.clear();
        return rVal;
    }
}
```

Κώδικας 14.16 *Η κλάση CardPlayer*

Η κλάση CardPlayer διαθέτει ArrayList<Card> στο οποίο αποθηκεύεται κάθε κάρτα που λαμβάνει ο παίκτης από τράπουλα. Τα αντικείμενα δημιουργούνται με κλήση στον δημιουργό της γονικής κλάσης Person. Η pickTop λαμβάνει μια κάρτα από την τράπουλα που της δίνεται παραμετρικά, η openCards αντιγράφει τις κάρτες που έχει ο παίκτης στα χέρια του σε ένα άλλο ArrayList, στη συνέχεια, διαγράφει τις κάρτες στα χέρια του παίκτη και επιστρέφει το αντίγραφο.

```
import java.time.LocalDate;

public class DicePlayer extends Person implements DicePlayerI {

    public DicePlayer(String id, String fName, String sName, LocalDate
birthday) {
        super(id, fName, sName, birthday);
    }

    @Override
    public int roll(Dice dice) {
        return dice.roll();
    }
}
```

Κώδικας 14.17 *Η κλάση DicePlayer*

Ο κώδικας 14.73 παρουσιάζει την κλάση παικτών ζαριού. Η κλάση Dice έχει δοθεί ήδη στον κώδικα 14.6. Η κλάση Person είναι η έκδοση που δίνεται στον κώδικα 12.19. Με τις κλάσεις αυτές στη διάθεσή μας μπορούμε να προχωρήσουμε στην ανάπτυξη της εφαρμογής.

```
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import java.util.Random;
import java.util.Set;

public class BeckersMeeting {

    private static final Map<DicePlayerI, Integer> diceScores = new
LinkedHashMap<>();
    private static final Dice fair = new Dice();
    private static final Dice two6 = new Dice(new double[]{1. / 6, 1. / 6,
1. / 6, 1. / 6, 0, 1. / 3,});
    private static final Dice two5 = new Dice(new double[]{1. / 6, 1. / 6,
1. / 6, 0, 1. / 3, 1. / 6,});
    private static final Dice[] dices = new Dice[]{fair, two6, two5};
    private final static Random gen = new Random();

    static Dice pickDice() {
        return dices[gen.nextInt(3)];
    }

    private static void diceGame(DicePlayerI p1, DicePlayerI p2) {
        int score1 = 0;
        int score2 = 0;
        for (int i = 0; i < 10; i++) {
            int s1 = p1.roll(pickDice());
            int s2 = p2.roll(pickDice());
            if (s1 > s2) {
                score1 += s1;
            } else if (s2 > s1) {
                score2 += s2;
            }
        }

        Integer score = diceScores.get(p1);
        if (score == null) {
            score = 0;
        }
        diceScores.put(p1, score1 + score);

        score = diceScores.get(p2);
        if (score == null) {
            score = 0;
        }
        diceScores.put(p1, score2 + score);
    }

    private static void cardGame(Set<CardPlayerI> cardPlayers) {
```

```
        Deck deck = new Deck();
        deck.shuffle();
        while (!deck.empty()) {
            for (CardPlayerI cp : cardPlayers) {
                cp.pickTop(deck);
                if (deck.empty()) {
                    break;
                }
            }
        }
    }

    private static void meet(Person p1, Person p2, Set<CardPlayerI>
cardPlayers) {
        if (p1 instanceof DicePlayerI && p2 instanceof DicePlayerI) {
            diceGame((DicePlayerI) p1, (DicePlayerI) p2);
        }

        if (p1 instanceof CardPlayerI) {
            cardPlayers.add((CardPlayerI) p1);
        }

        if (p2 instanceof CardPlayerI) {
            cardPlayers.add((CardPlayerI) p2);
        }
    }

    static int cntAces(ArrayList<Card> cards) {
        int rVal = 0;
        for (Card card : cards) {
            if (card.getCardRank() == CardRank.ACE) {
                rVal++;
            }
        }
        return rVal;
    }

    public static void main(String[] args) {
        Person[] persons = new Person[14];
        persons[0] = new DicePlayer("100", "Jack", "Antie",
LocalDate.of(2001, Month.MARCH, 1));
        persons[1] = new DicePlayer("101", "John", "Travolta",
LocalDate.of(1961, Month.AUGUST, 3));
        persons[2] = new DicePlayer("102", "Bill", "Allen",
LocalDate.of(1980, Month.FEBRUARY, 4));
        persons[3] = new DicePlayer("103", "Jim", "Morison",
LocalDate.of(2001, Month.JULY, 5));
        persons[4] = new DicePlayer("104", "Peter", "MacMillan",
LocalDate.of(2002, Month.MARCH, 1));
        persons[5] = new DicePlayer("105", "George", "Best",
LocalDate.of(1970, Month.MAY, 1));
        persons[6] = new DicePlayer("106", "George", "Blue",
LocalDate.of(1980, Month.MAY, 1));
        persons[7] = new DicePlayer("107", "Beny", "Hill",
LocalDate.of(1977, Month.JULY, 21));
        persons[8] = new CardPlayer("108", "John", "Beker",
LocalDate.of(1977, Month.JULY, 23));
        persons[9] = new CardPlayer("109", "Alison", "Stair",
LocalDate.of(1978, Month.APRIL, 22));
        persons[10] = new CardPlayer("110", "Erik", "Erikson",
LocalDate.of(1979, Month.JANUARY, 13));
```

```
        persons[11] = new Gambler("111", "Erik", "Johnson",
LocalDate.of(1989, Month.JANUARY, 13));
        persons[12] = new Gambler("112", "Klint", "Eastwood",
LocalDate.of(1983, Month.FEBRUARY, 12));
        persons[13] = new Gambler("113", "Jane", "Fonda",
LocalDate.of(1989, Month.JANUARY, 13));

        Set<CardPlayerI> cardPlayers = new HashSet<>();

        for (int i = 0; i < persons.length - 1; i++) {
            for (int j = i + 1; j < persons.length; j++) {
                meet(persons[i], persons[j], cardPlayers);
            }
        }

        cardGame(cardPlayers);

        List<Entry<DicePlayerI, Integer>> sortedScores = new
LinkedList<>(diceScores.entrySet());
        Collections.sort(sortedScores, new Comparator<Entry<DicePlayerI,
Integer>>() {
            @Override
            public int compare(Entry<DicePlayerI, Integer> o1,
Entry<DicePlayerI, Integer> o2) {
                return o2.getValue().compareTo(o1.getValue());
            }
        }
        );
       for (Entry<DicePlayerI, Integer> entry:sortedScores) {
           System.out.println(entry.getKey()+"->"+entry.getValue());
       }
       System.out.println("");

        Map<CardPlayerI, ArrayList<Card>> opened = new LinkedHashMap<>();
        int maxAces = 0;
        for (CardPlayerI p : cardPlayers) {
            ArrayList<Card> cardsInHand = p.openCards();
            System.out.print(p + ":\n ");
            System.out.println(cardsInHand);
            opened.put(p, cardsInHand);
            int cAces = cntAces(cardsInHand);
            if (cAces > maxAces) {
                maxAces = cAces;
            }
        }

        ArrayList<CardPlayerI> winners = new ArrayList<>();
        for (CardPlayerI p : opened.keySet()) {
            int cAces = cntAces(opened.get(p));
            if (cAces == maxAces) {
                winners.add(p);
            }
        }
        System.out.println("---------------------------------------");
        System.out.println("winner in card Game: \n" + winners + " Aces: "
+ maxAces);
    }
}
```

Κώδικας 14.18 *Η κλάση BeckersMeeting*

Ας ξεκινήσουμε από την main της κλάσης. Καταρχάς, δημιουργούμε και φορτώνουμε στον πίνακα persons τα 14 πρόσωπα. Τα 8 είναι DicePlayer, τα 3 CardPlayer και τα υπόλοιπα 3 Gambler. Στη συνέχεια, ορίζουμε το cardPlayers τύπου HashSet. Το cardPlayers είναι ο κατάλογος στον οποίο εγγράφονται οι χαρτοπαίκτες σύμφωνα με την εκφώνηση. Η επιλογή του HashSet βασίστηκε στο γεγονός ότι κάθε χαρτοπαίκτης έχει νόημα να γραφτεί μόνο μία φορά. Αμέσως μετά υλοποιείται η απαίτηση να συναντηθούν τα πρόσωπα ανά δύο με τη βοήθεια της meet. Προσέξτε την εμφωλευμένη for μέσα στην οποία καλείται η meet. Οι συνδυασμοί ν αντικειμένων ανά 2 είναι ν×((ν-1))/2 , αυτό ακριβώς αποδίδει η for. Με άλλα λόγια, για κάθε ζεύγος προσώπων p1, p2 συναντάται το ζεύγος (p1, p2) μόνον εφόσον p1 διάφορο του p2, ενώ δεν επαναλαμβάνεται συνάντηση για το ζεύγος (p2, p1).

Αν μεταφερθούμε προσωρινά στη συνάρτηση meet, βλέπουμε πως καταρχάς ελέγχεται αν είναι και τα δύο πρόσωπα παίκτες ζαριού. Στην περίπτωση αυτή παίζουν μια παρτίδα ζάρια με την diceGame. Στη συνέχεια εφόσον είναι κάποιο από τα πρόσωπα χαρτοπαίκτης γίνεται η εγγραφή του στην ομάδα χαρτοπαικτών. Με το τέλος των συναντήσεων, διεξάγεται η παρτίδα χαρτιών με την cardGame.

Η diceGame ενημερώνει με το σκορ όλων των παιχνιδιών κάθε παίκτη την diceScores που είναι στατική μεταβλητή της κλάσης BeckersMeeting. Με την cardGame, ο κάθε χαρτοπαίκτης διατηρεί μια σειρά από κάρτες με βάση τις οποίες βγαίνουν οι νικητές.

Στη συνέχεια της main έχουμε να εμφανίσουμε τα αποτελέσματα. Θα πρέπει καταρχάς να ταξινομήσουμε τα αποτελέσματα των παιχνιδιών ζαριού ως προς φθίνουσα σειρά των σκορ. Εδώ μας βοηθάει η λίστα sortedScores. Κάθε στοιχείο αυτής της λίστας αναπαριστά ένα στοιχείο της diceScores. Στη συνέχεια ταξινομείται η λίστα με τη βοήθεια του Comparator που περνάμε ως παράμετρο στην Collections.sort. Προσέξτε πως ο Comparator δημιουργείται ως αντικείμενο ανώνυμης κλάσης. Επίσης, σημειώστε πως η compare του Comparator ταξινομεί κατά φθίνουσα σειρά. Έχοντας ταξινομημένα τα scores είναι εύκολο να τα εμφανίσουμε κατά φθίνουσα σειρά σύμφωνα με τις απαιτήσεις της εκφώνησης.

Τώρα θα πρέπει να ανοίξουν οι χαρτοπαίκτες τα χαρτιά που έχει ο κάθε ένας στο χέρι του, να γίνει καταμέτρηση και να ανακηρυχτεί ο νικητής ή οι νικητές. Οπότε κάθε ένας εγγεγραμμένος χαρτοπαίκτης ανοίγει τις κάρτες του που προσωρινά αποθηκεύονται στην τοπική μεταβλητή cardsInHand. Στη συνέχεια εμφανίζονται τα στοιχεία του παίκτη και τα χαρτιά του. Επιπλέον, τα χαρτιά κάθε παίκτη εισάγονται στην opened καθώς χρειάζονται για την τελική καταμέτρηση και ανάδειξη των νικητών. Παράλληλα με όλη αυτή τη διαδικασία υπολογίζεται ο μέγιστος αριθμός άσων που υπάρχουν στα χαρτιά κάθε χαρτοπαίκτη. Όλοι όσοι έχουν τον ίδιο αριθμό άσων θα πρέπει να ανακηρυχτούν νικητές.

Τέλος, εντοπίζονται οι νικητές της χαρτοπαιξίας και εμφανίζονται τα στοιχεία τους.

14.5.3 Πίνακας χημικών στοιχείων

Ο περιοδικός πίνακας των χημικών στοιχείων [3] περιλαμβάνει όλα τα γνωστά χημικά στοιχεία κατηγοριοποιημένα σε ομάδες, περιόδους και τομείς. Πιο συγκεκριμένα, υπάρχουν 18 ομάδες, 7 περίοδοι και 4 τομείς. Οι ομάδες αριθμούνται από 1 έως 18, οι περίοδοι από 1 έως 7, ενώ οι τομείς χαρακτηρίζονται με τους χαρακτήρες s, p, d, f. Κάθε χημικό στοιχείο ανήκει σε έναν τομέα, σε μία ομάδα και σε μία περίοδο. Επιπλέον, κάθε χημικό στοιχείο έχει ένα σύμβολο αναφοράς, π.χ. Η για το υδρογόνο, Na για το Νάτριο, καθώς και έναν ατομικό αριθμό.

Να αναπτυχθεί η κλάση ChemicalElements κάθε αντικείμενο της οποίας αναπαριστά ένα χημικό στοιχείο. Επιπλέον, να αναπτυχθεί η κλάση TableOfChemichalElements που αναπαριστά πίνακες χημικών στοιχείων και διαθέτει τις μεθόδους:

```
public boolean add(ChemicalElement cE)
```

Αν το στοιχείο cE, υπάρχει στον πίνακα επιστρέφει false. Αν δεν υπάρχει, το εισάγει και επιστρέφει true.

```
public TableOfChemichalElements getGroup(int groupId)
```

Επιστρέφει έναν πίνακα που περιλαμβάνει τα στοιχεία αυτού του πίνακα που ανήκουν στην ομάδα groupId.

```
public TableOfChemichalElements getPeriod(int periodId)
```

Επιστρέφει έναν πίνακα που περιλαμβάνει τα στοιχεία αυτού του πίνακα που ανήκουν στην περίοδο periodId.

```
public TableOfChemichalElements getSection(char sectionId)
```

Επιστρέφει έναν πίνακα που περιλαμβάνει τα στοιχεία αυτού του πίνακα που ανήκουν στον τομέα sectionId.

```
public boolean isEmpty()
```

Ελέγχει αν αυτός ο πίνακας είναι κενός.

```
public Iterator groupIterator(int groupId)
```

Επιστρέφει έναν Iterator στα στοιχεία της ομάδας με groupId. Ο Iterator υποστηρίζει εκτός από την hasNext και next και την remove. Η remove διαγράφει το τελευταίο στοιχείο που επέστρεψε η next. Δείτε τη διεπαφή της remove στην τεκμηρίωση του Iterator.
Προσθέστε main με βασικό τεστ κώδικα της κλάσης TableOfChemichalElements.

Λύση
Στον κώδικα 14.19, παρατίθεται η κλάση ChemicalElement.

```
public class ChemicalElement {

    private final String symbol;
    private final int atomicNumber;
    private final int group;
    private final int period;
    private final char section;

    public ChemicalElement(int group, int period, char sector, int
atomicNumber, String symbol) {
        this.group = group;
        this.period = period;
        this.section = sector;
        this.atomicNumber = atomicNumber;
        this.symbol = symbol;
    }

    public int getGroup() {
        return group;
    }

    public int getPeriod() {
        return period;
    }

    public char getSection() {
        return section;
    }

    public int getAtomicNumber() {
        return atomicNumber;
    }

    public String getSymbol() {
        return symbol;
    }

    @Override
    public int hashCode() {
        int hash = 7;
        hash = 89 * hash + this.symbol.hashCode();
        return hash;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
```

```
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final ChemicalElement other = (ChemicalElement) obj;
        return this.symbol.equals(other.symbol);
    }

    @Override
    public String toString() {
        return symbol + ", AN=" + atomicNumber + ", group=" + group + ",
period=" + period + ", sector=" + section;
    }
}
```

Κώδικας 14.19 *Η κλάση ChemicalElement*

Η κλάση ChemicalElement είναι τυπική κλάση και περικλείει όλες τις αναγκαίες μεθόδους. Στον κώδικα 14.20 παρατίθεται η κλάση TableOfChemichalElements.

```
import java.util.ArrayList;
import java.util.Iterator;

public class TableOfChemichalElements {

    private final ArrayList<ChemicalElement> store = new ArrayList<>();

    public class GroupIterator implements Iterator {

        private final int groupId;
        private int current = 0;

        public GroupIterator(int groupId) {
            this.groupId = groupId;
        }

        @Override
        public boolean hasNext() {

            while (current < store.size()) {
                ChemicalElement cE = store.get(current);
                if (cE.getGroup() == groupId) {
                    return true;
                }
                current++;
            }
            return false;
        }

        @Override
        public Object next() {
            return store.get(current++);
        }

        @Override
        public void remove() {
            store.remove(--current);
```

```
        }
    }

    public Iterator groupIterator(int groupId) {
        return new GroupIterator(groupId);
    }

    public boolean add(ChemicalElement cE) {
        if (store.contains(cE)) {
            return false;
        }
        return store.add(cE);
    }

    public TableOfChemichalElements getGroup(int groupId) {
        TableOfChemichalElements rVal = new TableOfChemichalElements();
        for (ChemicalElement cE : store) {
            if (cE.getGroup() == groupId) {
                rVal.add(cE);
            }
        }
        return rVal;
    }

    public TableOfChemichalElements getPeriod(int periodId) {
        TableOfChemichalElements rVal = new TableOfChemichalElements();
        for (ChemicalElement cE : store) {
            if (cE.getPeriod() == periodId) {
                rVal.add(cE);
            }
        }
        return rVal;
    }

    public TableOfChemichalElements getSection(char sectionId) {
        TableOfChemichalElements rVal = new TableOfChemichalElements();
        for (ChemicalElement cE : store) {
            if (cE.getSection() == sectionId) {
                rVal.add(cE);
            }
        }
        return rVal;
    }

    public boolean isEmpty() {
        return store.isEmpty();
    }

    @Override
    public String toString() {
        StringBuilder t = new StringBuilder();
        for (ChemicalElement cE : store) {
            t.append(cE).append("\n");
        }
        return t.toString();
    }

    public static TableOfChemichalElements setUp() {
        ChemicalElement h = new ChemicalElement(1, 1, 's', 1, "H");
        ChemicalElement li = new ChemicalElement(1, 2, 's', 3, "Li");
        ChemicalElement na = new ChemicalElement(1, 3, 's', 11, "Na");
```

```
            ChemicalElement be = new ChemicalElement(2, 2, 's', 4, "Be");
            ChemicalElement mg = new ChemicalElement(2, 3, 's', 12, "Mg");
            ChemicalElement sc = new ChemicalElement(3, 4, 'd', 21, "Sc");
            ChemicalElement y = new ChemicalElement(3, 5, 'd', 39, "Y");
            ChemicalElement he = new ChemicalElement(18, 1, 'p', 2, "He");
            ChemicalElement la = new ChemicalElement(3, 6, 'f', 57, "La");
            TableOfChemichalElements gen = new TableOfChemichalElements();
            gen.add(h);
            gen.add(li);
            gen.add(na);
            gen.add(be);
            gen.add(mg);
            gen.add(sc);
            gen.add(y);
            gen.add(he);
            gen.add(la);
            return gen;
        }

        public static void main(String[] args) {

            TableOfChemichalElements gen = setUp();
            System.out.println(gen);
            TableOfChemichalElements group1 = gen.getGroup(1);
            System.out.println(group1);
            System.out.println(gen.getPeriod(1));
            System.out.println(gen.getSection('p'));

            System.out.println("----------------------------");
            Iterator it = gen.groupIterator(1);
            while (it.hasNext()) {
                ChemicalElement cE = (ChemicalElement) it.next();
                if (cE.getPeriod() == 2) {
                    it.remove();
                }
                System.out.println(cE);
            }
            System.out.println("----------------------------");
            System.out.println(gen);
        }
    }
```

Κώδικας 14.20 *Η κλάση TableOfChemichalElements*

Οι getGroup, getPeriod και getSection είναι προφανείς. Προκειμένου να υλοποιηθεί η getGroupIterator χρησιμοποιείται η εσωτερική κλάση GroupIterator η οποία υλοποιεί τη διεπαφή Iterator. Ο δημιουργός της κλάσης λαμβάνει ως παράμετρο το groupId της ομάδας του Iterator και ενημερώνει την αντίστοιχη μεταβλητή στιγμιότυπου. Επίσης, η ιδιωτική μεταβλητή current χρησιμεύει ως δείκτης στον πίνακα store της TableOfChemichalElements. Η hasNext αυξάνει την current έως ότου βρει στη θέση που υποδεικνύει η current στον πίνακα store στοιχείο που ανήκει στην ομάδα με groupId. Αν το βρει, επιστρέφει true διαφορετικά false. Η next επιστρέφει το στοιχείο στη θέση current του store και αυξάνει την current κατά 1. Η remove διαγράφει το στοιχείο που επέστρεψε η τελευταία κλήση της next. Ωστόσο, η next αφού επιστρέψει το στοιχείο αυξάνει την current κατά 1. Επομένως, η remove για να διαγράψει το σωστό στοιχείο θα πρέπει να μειώσει την current κατά 1. Εξάλλου με τη διαγραφή ενός στοιχείου μειώνεται και το μέγεθος του store κατά 1.

Έχοντας την κλάση GroupIterator, η συνάρτηση getGroupIterator επιστρέφει απλώς ένα αντικείμενο GroupIterator. Η setUp είναι στατική συνάρτηση που απλώς φορτώνει μερικά στοιχεία σε κατάλληλο ArrayList και τα επιστρέφει. Επίσης, κατάλληλα έχει υπερφορτωθεί και η toString ώστε να τυπώνει τα στοιχεία του πίνακα ένα ανά γραμμή βασιζόμενη στην toString της ChemicalElement.

Στην main φορτώνουμε με τη βοήθεια της setUp τον πίνακα gen, στη συνέχεια ελέγχουμε τις getGroup, getPeriod και getSection. Τέλος, ελέγχεται ο Iterator.